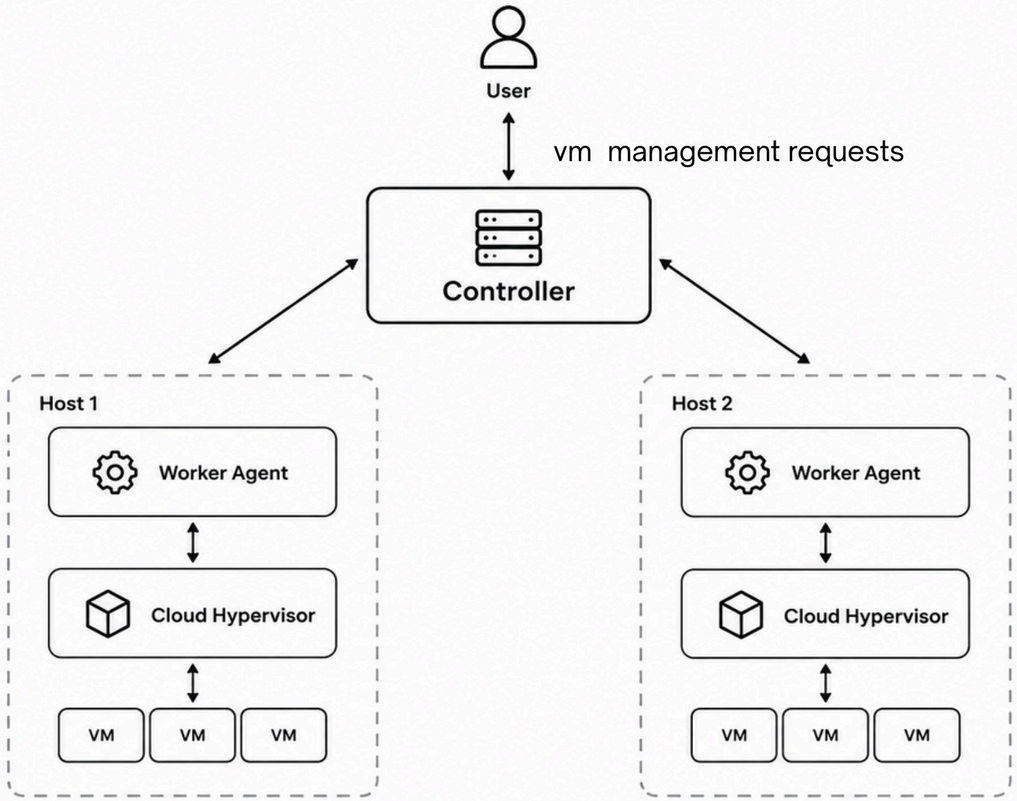
Minimal Two-Node Cloud Hypervisor VM Controller

2026

ps-023

A minimal distributed VM management control plane using Cloud Hypervisor across two KVM-capable compute hosts.

- Shivani S R
  - shivani.s.raravi@gmail.com
- Rajya vardhan .M
- rajyavardhan200@gmail.com



WITH SINCERE GRATITUDE

THANK YOU FOR GIVING US THIS OPPORTUNITY

"To the Organizers, Mentors & Jury: thank you for challenging us to build real systems."

From bare-metal KVM to 87 passing tests — an unforgettable engineering and learning journey.

THE OPPORTUNITY

To the Hackathon Organizers

AIORI 2026

"A stage for authentic systems engineering."

- **Challenging PS-023 Track**
  Pushed us to build real low-level systems
- **Freedom to Innovate**
  Direct bare-metal KVM & Cloud Hypervisor platform
- **Flawless Organization**
  Deeply grateful for this inspiring hackathon stage

With Deepest Respect • AIORI Organizers

THE MENTORSHIP

To our Mentors & Jury

WISDOM & GUIDANCE

"First principles over quick shortcuts."

- **Sharp Technical Feedback**
  Pushed us to eliminate all VM process leaks
- **Uncompromising Standards**
  Inspired our 4-stage stop escalation engine
- **Valued Time & Review**
  Deeply thankful for your guidance and insights

With Heartfelt Thanks • Mentors & Jury

THE LEARNING JOURNEY

To our Teammates & Peers

RESILIENCE & CRAFT

"Built with curiosity, discipline & heart."

- **Relentless Problem-Solving**
  Late-night KVM ioctls & socket debugging
- **87 / 87 Verified Tests**
  100% test pass rate across both physical nodes
- **Lifelong Takeaways**
  Rediscovered the pure joy of software craft

With Appreciation • Team TC246

RAJYAVARDHAN & TEAM TC246

Cloud Computing Track • Problem Statement PS-023

WITH HEARTFELT APPRECIATION

"Grateful for the opportunity to learn, build, and grow."

THANK YOU FOR YOUR TIME

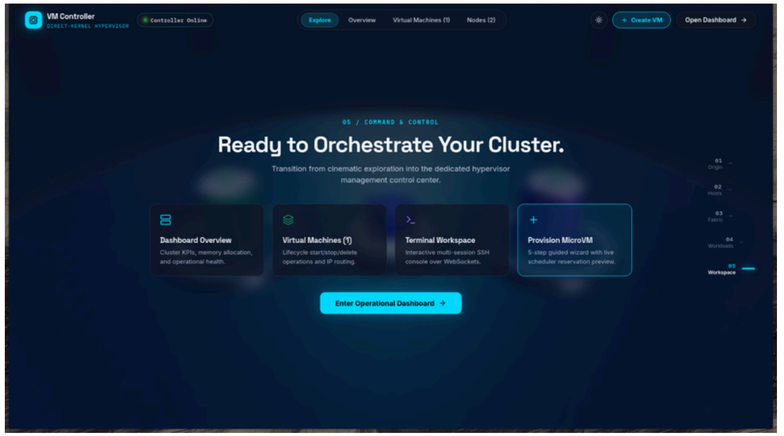
Honored to present • Open for Conversation & Q&A

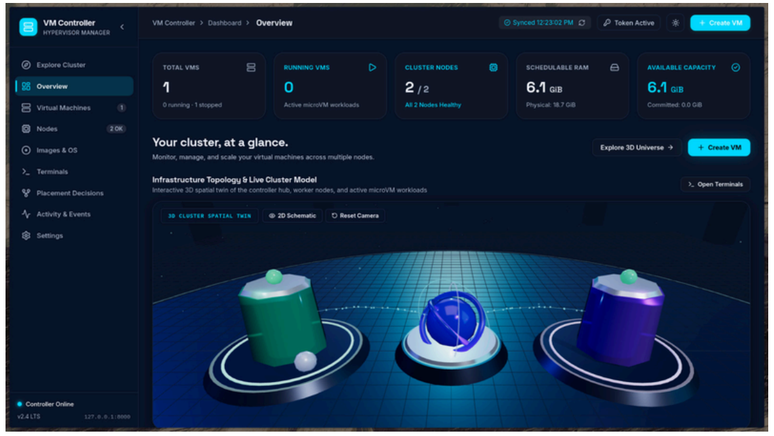
Introduction

Lightweight Cloud Virtualisation

Cloud computing relies on virtualisation for flexible multi-host resource allocation. Coordinated management via a centralised control plane simplifies VM lifecycle operations across compute nodes.

- Coordinated VM Control
- Lightweight Hypervisor
- Two-Node Orchestration

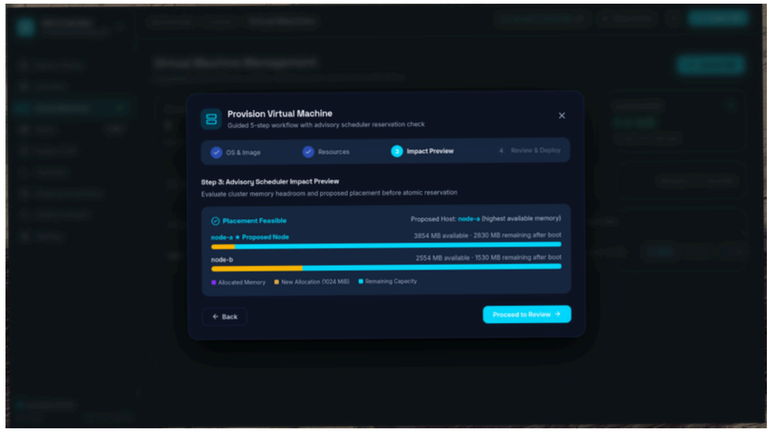




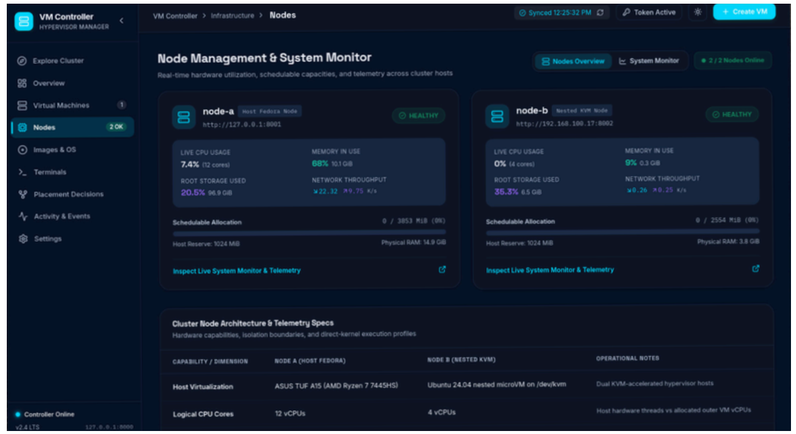
The Problem

Multi-Host Coordination Challenges

Managing VMs independently across hosts increases operational complexity. Centralized coordination must handle memory placement, resource limits, and prevent orphaned VMM processes upon failure.



Memory-Aware Placement



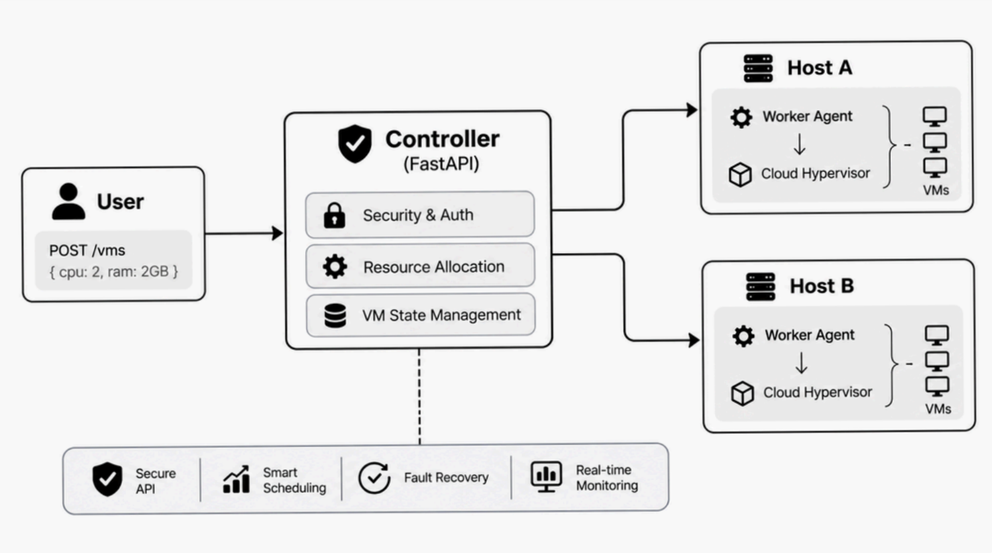
Resource-Limit Enforcement

Objectives

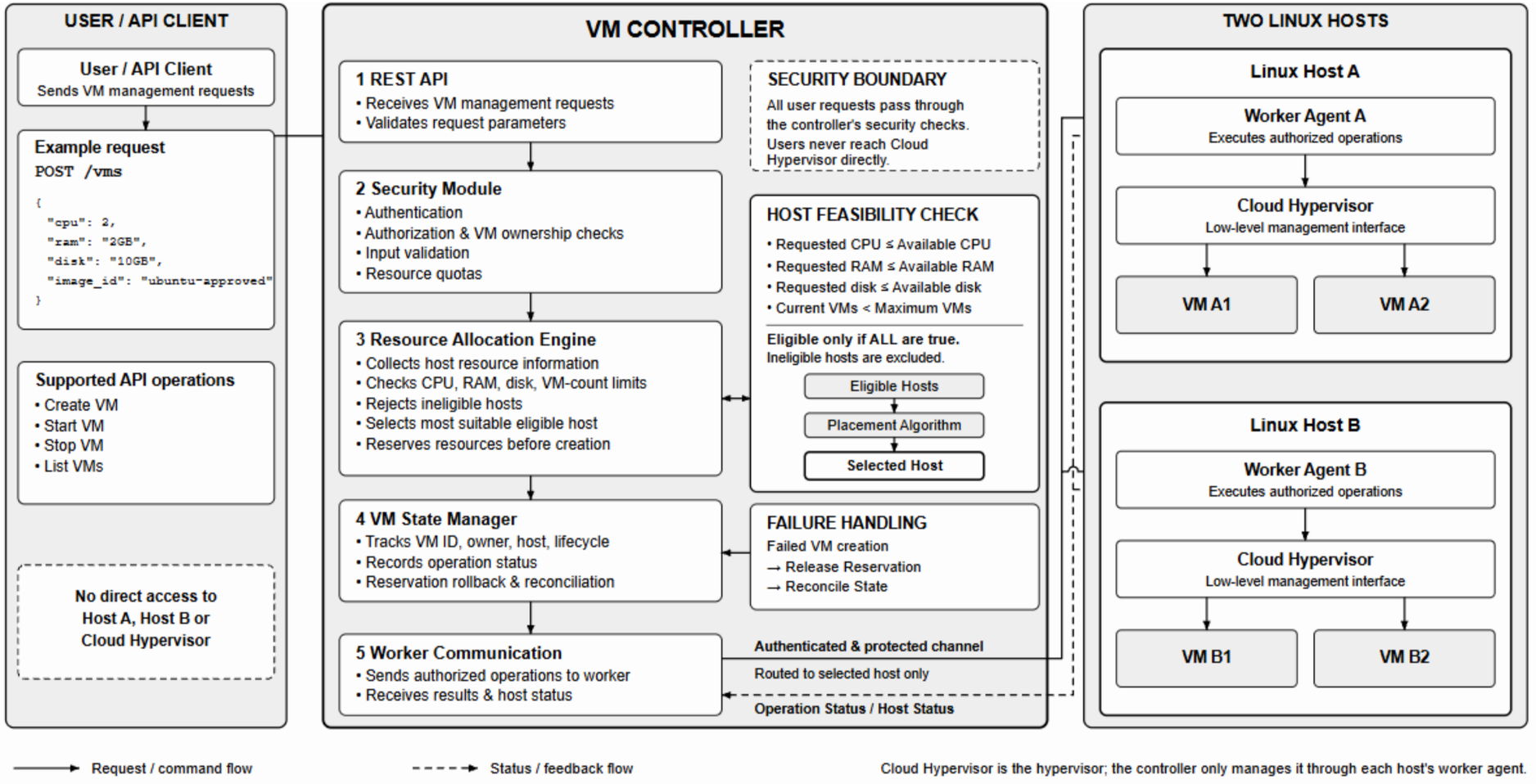
Control Plane Goals & Runtime Telemetry

A systems virtualization project delivering a centralized controller for two KVM workers via Cloud Hypervisor APIs, taking VM configs (vCPU, RAM, image) and node telemetry to manage VM lifecycles.

- Lifecycle & Placement
- Host & Telemetry Inputs
- Failure Safety & Health



Implementation

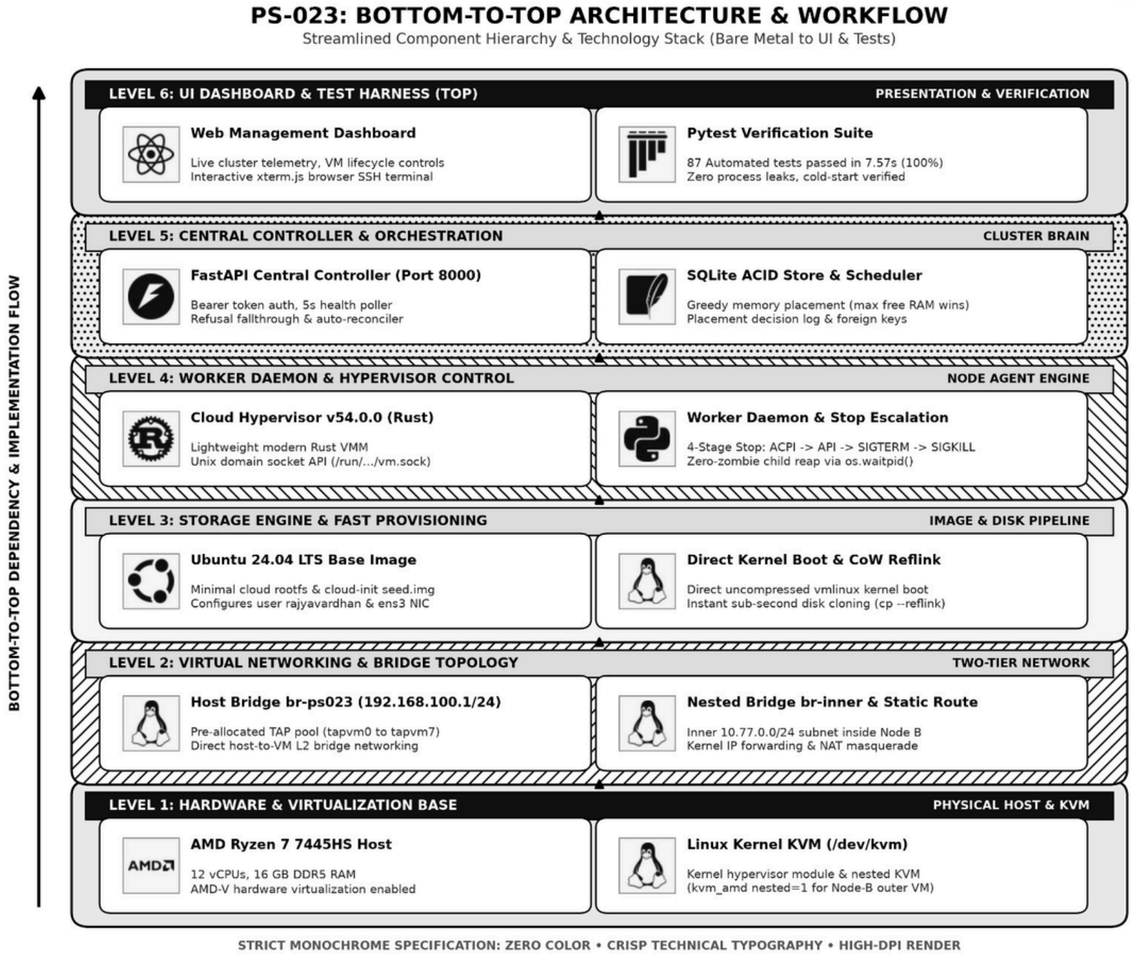


Project Roadmap

Next Course of Action & Future Work

Focusing next on full lifecycle testing across both workers, resource exhaustion validation, and failure recovery; followed by planned research extensions into resource-aware scheduling.

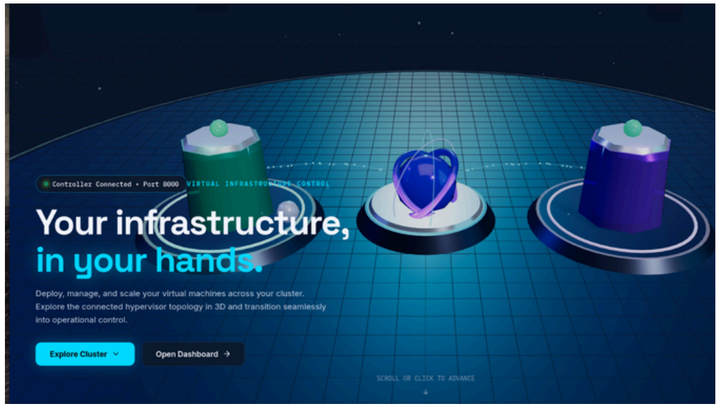
- Lifecycle Validation
- Failure & Cleanup
- Resource Schedulers

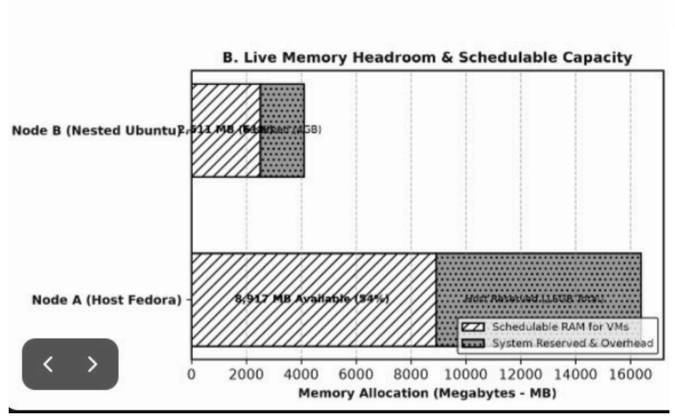


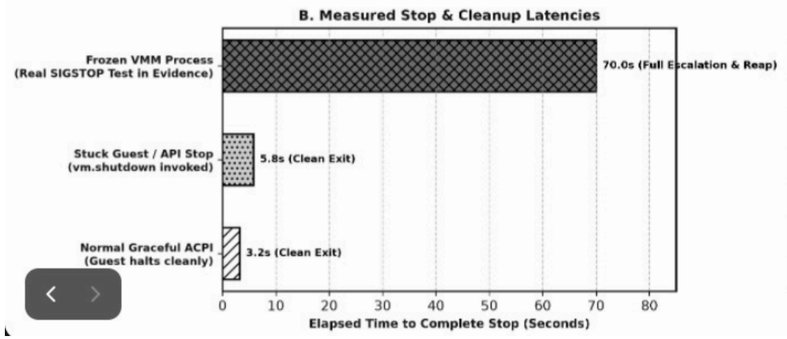
Current Progress

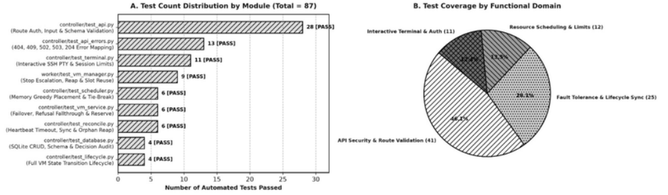
87 Automated Tests Passed

Confirmed test suites verify controller, scheduler, and worker lifecycle pipelines under pytest. Cluster bring-up and failure-injection tests recorded. Performance evaluation pending.









High-Level Design

SYSTEM COMMUNICATION FLOW

Client → Controller → Scheduler → Selected Worker → Cloud Hypervisor → KVM → VM

Communication flow: Client -> Controller -> Scheduler -> Selected Worker -> Cloud Hypervisor -> KVM -> VM.
Authenticated REST and Unix sockets coordinate operations.

Client

Sends VM Requests

Initiates lifecycle requests & secure user sessions

CLIENT LAYER

REST API

Controller

API & Scheduler

Request authentication, routing & worker scheduling

CONTROL PLANE

REST / Auth

Workers A & B

Host & Lifecycle

Host compute daemon, state tracking & process isolation

COMPUTE TIER

Unix Sockets

Cloud Hypervisor

KVM Virtualisation

Rust-based VMM interfacing with Linux KVM ioctls

VMM & KVM